
MISSALPHA AI LAB
Top 10
Life-Changing
AI Prompts
By MissAlpha ai lab
1. AI Life Mentor
(The Ultimate Prompt)
LEARN MORE
1. AI Life Mentor
(The Ultimate Prompt)
Act as my personal AI life mentor.
Help me with my goals, productivity,
habits, mindset, career, health, relationships,
and give me step-by-step guidance to
become the best version of myself.
Goals
Productivity
Habits
Mindset
Career
Health
Relationships

MissIpha_

MISSALPHA™ AI-Powered-Development

AI Coding Agents Masterclass —

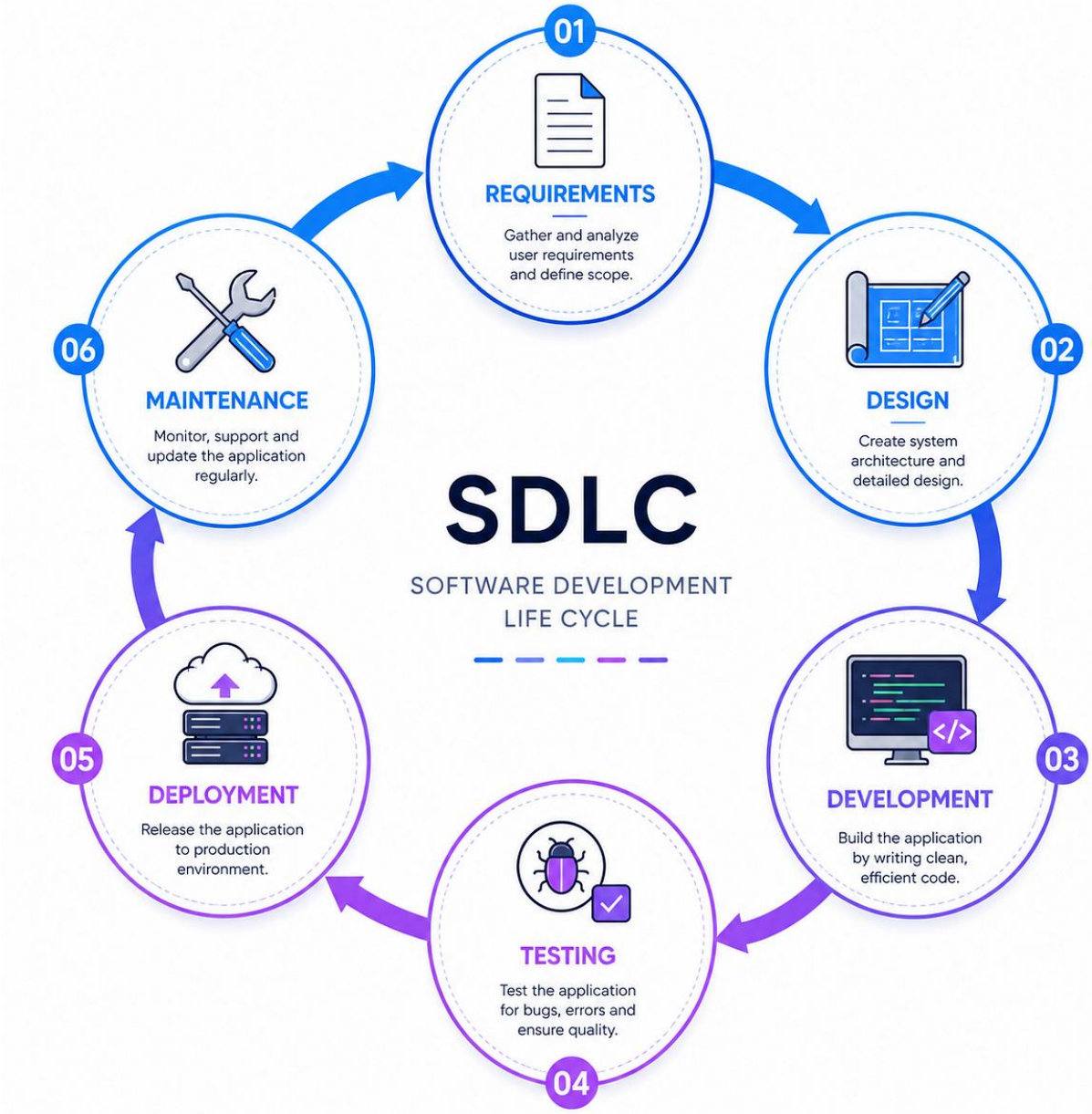
One project → one workflow → one tool for each responsibility.

3 LIVE BUILD CHALLENGES

One SDLC. Three AI Coding Workflows.

Task	Project	Tool	Main Learning
01	StyleAura Clothing Store	Bolt	AI → App from idea to deployment
02	StyleAura E-Commerce	VS Code + GitHub Copilot	Professional developer + AI + Git
03	MISSALPHA™ AI-Powered-PA	Cursor AI	AI Agent + full-stack development

Missalpha_AILab


01
REQUIREMENTS
Gather and analyze user requirements and define scope.
02
DESIGN
Create system architecture and detailed design.
03
DEVELOPMENT
Build the application by writing clean, efficient code.
04
TESTING
Test the application for bugs, errors and ensure quality.
05
DEPLOYMENT
Release the application to production environment.
06
MAINTENANCE
Monitor, support and update the application regularly.
SDLC
SOFTWARE DEVELOPMENT LIFE CYCLE

TASK 01 — StyleAura™ Fashion Website

Tool: Bolt

Level: Beginner

Goal: Idea → UI → Working Website

Backend: ❌

Database: ❌

Authentication: ❌

Payment: ❌

API: ❌

Purpose: Demonstrate how AI can create and customize a professional website with minimal effort.

PROMPT

Build a simple, modern, responsive fashion website called "StyleAura". StyleAura is a modern clothing brand for young professionals.

Create a SINGLE-PAGE website only.

Sections:

1. Navbar

- *StyleAura logo/text*
- *Home*
- *Collections*
- *About*

Misslpha_AILap

- Contact

2. Hero Section

- Heading: "Style That Defines You"

- Short fashion-focused tagline

- Primary CTA: "Explore Collection"

- Attractive fashion background/image

3. Featured Collection

Display 6 clothing products in clean cards.

Each card should contain:

- Product image

- Product name

- Category

- Price

Misslpha_AILap

- "Shop Now" button

Use these products:

- Classic Black Dress

- Urban Denim Jacket

- Essential White Shirt

- Premium Blue Jeans

- Elegant Beige Top

- Casual Street Hoodie

4. About StyleAura

Add a short section explaining that StyleAura combines

modern fashion, comfort and confidence.

5. Why StyleAura

Missipha_AILap

Show 3 simple benefits:

- Premium Style

- Comfortable Fashion

- Affordable Prices

6. Contact / CTA

Heading: "Ready to Upgrade Your Style?"

Button: "Explore Collection"

7. Footer

- StyleAura

- © 2026 StyleAura

- Instagram / Facebook / LinkedIn text links

DESIGN:

Missipha_AILap

- Premium but simple fashion aesthetic

- Clean white/light background

- Black typography

- Elegant accent color

- Large fashion imagery

- Modern cards

- Rounded buttons

- Good spacing

- Professional typography

- Fully responsive for desktop, tablet and mobile

TECHNICAL REQUIREMENTS:

- Use React + Vite.

- Use Tailwind CSS if available.

- Keep the project simple and lightweight.

Missipha_AILap

- No backend.

- No database.

- No authentication.

- No payment gateway.

- No external API.

- Use static/local product data.

- Use reusable React components.

- Add smooth scrolling between sections.

- Make all navigation and CTA buttons work.

- Ensure there are no console errors.

IMPORTANT:

Do not over-engineer the application.

Do not create unnecessary files or features.

Focus on producing a polished, professional single-page website that can be demonstrated in a live AI Coding Agents masterclass.

Missipha_AILap

After implementation:

1. Run the application.

2. Check the page visually.

3. Fix any errors.

4. Verify responsive behavior.

5. Confirm the final website is ready to demonstrate.

After Bolt generates the website, demonstrate the power of iterative prompting:

Modification 1 — Branding

Make the StyleAura website look more premium and fashion-focused.

Improve the typography, spacing, hero section, product cards and CTA buttons.

Keep the existing structure and functionality.

Do not add new features.

Modification 2 — Mobile

Optimize the entire StyleAura website for mobile devices.

Fix spacing, typography, navigation, product cards and image sizing.

Missipha_AILap

Do not change the content or add new features.

Modification 3 — Final QA

Act as a professional QA engineer.

Review the entire StyleAura website for:

- Broken navigation

- Non-working buttons

- Responsive issues

- Layout problems

- Console errors

- Accessibility issues

- Poor spacing or typography

Fix only the issues you find.

Do not add new features or change the existing business requirements.

Missipha_AILap

IDEA → PROMPT → AI GENERATION → REVIEW → MODIFY → TEST → DEPLOY

TASK 02

STYLEAURA™ E-COMMERCE

VS CODE + GITHUB COPILOT + GIT/GITHUB

This is where you move from:

> **"AI builds my application."**

to:

> **"I am a developer using AI inside a professional SDLC."**

GitHub Copilot currently supports **Ask, Plan and Agent modes** in supported IDE workflows; Agent mode can inspect files, edit multiple files, run commands and iterate, while the developer remains responsible for review and approval. GitHub Docs

---

TECHNOLOGY

Frontend

React + Vite

MisAlpha_AILab

Backend

Python + FastAPI

Database

SQLite

AI

GitHub Copilot

IDE

VS Code

Version Control

Git + GitHub

STEP 0 — GIT SETUP

Before Copilot writes code:

```
mkdir styleaura-ecommerce
```

```
cd styleaura-ecommerce
```

```
git init
```

Misslpha_AILap

`git branch -M main`

`git checkout -b develop`

Create GitHub repository:

`styleaura-ecommerce`

Recommended branches:

`main`

`develop`

`feature/frontend`

`feature/product-api`

`feature/cart`

`feature/checkout`

Your deck emphasizes branch isolation, diff review, testing and human approval rather than allowing AI changes directly into `main`. Master-Class-14

SDLC 01 — REQUIREMENTS

COPILOT PROMPT 01

Use Plan mode first.

Act as a Senior Product Manager and Business Analyst.

PROJECT:

Misslpha_AILap

StyleAura™ E-Commerce Platform.

BUSINESS:

StyleAura™ is a clothing e-commerce business.

TECHNOLOGY:

Frontend:

React + Vite

Backend:

Python + FastAPI

Database:

SQLite

Version Control:

Git + GitHub

OBJECTIVE:

Build a maintainable full-stack e-commerce application.

CORE USER FEATURES:

Missipha_AILap

1. Browse products

2. Search products

3. Filter products

4. View product details

5. Add to cart

6. Update quantity

7. Remove items

8. Checkout

9. Create order

10. View order confirmation

PRODUCT CATEGORIES:

Dresses

Tops

Jeans

Jackets

Accessories

DO NOT IMPLEMENT YET.

Missipha_AILap

Analyze the requirements.

Return:

1. Business requirements
2. Functional requirements
3. Non-functional requirements
4. User stories
5. Acceptance criteria
6. MVP scope
7. Risks
8. Dependencies
9. Recommended implementation order

Do not guess requirements.

If something is unclear, identify it as an assumption.

Missipha_AILab

SDLC 02 — DESIGN

PROMPT 02 — SYSTEM ARCHITECTURE

Act as a Senior Full-Stack Software Architect.

Design the StyleAura™ e-commerce system.

STACK:

React

FastAPI

SQLite

REST API

ARCHITECTURE:

React Frontend

↓

REST API

↓

FastAPI Backend

↓

Service Layer

↓

Database Layer

Missipha_AILap

↓

SQLite

DESIGN:

Define:

1. Frontend architecture

2. Backend architecture

3. Database architecture

4. API architecture

5. Folder structure

6. Component structure

7. Data flow

8. Error handling strategy

9. Validation strategy

10. Testing strategy

11. Security strategy

DATABASE ENTITIES:

Product

Missipha_AILap

Category

Cart

CartItem

Order

OrderItem

For each entity define:

- Primary key
- Fields
- Data types
- Relationships
- Constraints

API DESIGN:

Define endpoints for:

Products

Cart

Orders

Misslpha_AILap

Do not implement yet.

Return the architecture as a technical design document.

PROMPT 03 — DATABASE DESIGN

> *Act as a Senior Database Engineer.*

Implement only the database layer for StyleAura™.

DATABASE:

SQLite

TABLES:

categories

products

carts

cart_items

Misslpha_AILap

orders

order_items

Requirements:

- Use primary keys.

- Use foreign keys.

- Add appropriate constraints.

- Product price must be positive.

- Quantity must be greater than zero.

- OrderItem must preserve purchase price.

- Avoid unnecessary duplication.

- Maintain relational integrity.

First inspect the existing repository.

Do not modify unrelated files.

Create:

1. Database configuration

Missipha_AILap

2. Models

3. Relationships

4. Database initialization

5. Database tests

After implementation:

Run tests.

Report:

Files changed

Schema created

Tests executed

Test results

Potential risks

Misslpha_AILap

SDLC 03 — DEVELOPMENT

PROMPT 04 — BACKEND PRODUCT API

Act as a Senior Python FastAPI Developer.

Implement the Product API.

Existing database:

products

categories

Endpoints:

GET /api/products

GET /api/products/{id}

POST /api/products

PUT /api/products/{id}

DELETE /api/products/{id}

Requirements:

- Request validation

- Response validation

- Correct HTTP status codes

- Product-not-found handling

- Database error handling

- Clean separation between routes and business logic

- Reusable services

- Automated tests

Before modifying:

Missipha_AILap

Inspect the existing architecture.

Do not rewrite unrelated files.

After implementation:

1. Run backend tests.

2. Test every endpoint.

3. Test invalid input.

4. Test missing product.

5. Report results.

Do not mark complete until tests pass.

PROMPT 05 — REACT FRONTEND

Act as a Senior React Engineer.

Implement the StyleAura™ product frontend.

Missipha_AILap

Create:

ProductList

ProductCard

ProductDetails

SearchBar

CategoryFilter

LoadingState

ErrorState

EmptyState

Requirements:

- Fetch data from FastAPI.
- Do not hardcode API responses.
- Use reusable components.
- Handle loading.
- Handle API failure.
- Handle empty results.
- Make responsive.
- Keep API configuration in environment variables.

Missipha_AILap

Do not modify unrelated backend code.

After implementation:

1. Start frontend.

2. Start backend.

3. Verify API connection.

4. Test product listing.

5. Test search.

6. Test filtering.

7. Test product details.

Report test results.

PROMPT 06 — CART FEATURE

Implement the StyleAura™ shopping cart.

USER STORIES:

Missipha_AILap

As a customer, I want to add products to my cart

so that I can purchase them.

As a customer, I want to update quantity

so that I can control how many products I purchase.

As a customer, I want to remove products

so that I can correct my order.

FEATURES:

- Add to cart
- View cart
- Increase quantity
- Decrease quantity
- Remove product
- Calculate subtotal
- Calculate total

CONSTRAINTS:

Missipha_AILap

- Quantity must be greater than zero.

- Do not duplicate product records.

- Reuse existing API architecture.

- Add validation.

- Add tests.

Before coding:

Inspect existing cart-related files.

Then:

Plan

→ Implement

→ Test

→ Report

PROMPT 07 — CHECKOUT + ORDER

Implement the checkout and order workflow.

USER FLOW:

Misslpha_AILap

Cart

↓

Checkout

↓

Customer Information

↓

Order Review

↓

Place Order

↓

Order Confirmation

Customer fields:

name

email

phone

address

Requirements:

Missipha_AILap

1. Validate customer information.

2. Validate cart.

3. Create order.

4. Create order items.

5. Preserve product price at purchase time.

6. Calculate order total.

7. Return order confirmation.

8. Clear cart after successful order.

DO NOT implement real payment processing.

Use a simulated order confirmation.

Add:

- Backend validation

- Frontend validation

- API tests

- Integration tests

Do not modify unrelated features.

Misslpha_AILap

GIT CHECKPOINT

After each major feature:

```
git status
```

```
git diff
```

```
git add .
```

```
git commit -m "feat: add product catalog"
```

```
git push origin develop
```

Then create:

```
feature/product-catalog
```

or the appropriate feature branch.

SDLC 04 — TESTING

PROMPT 08 — FULL QA

```
Act as a Senior QA Engineer.
```

Review the entire StyleAura™ full-stack application.

Create a test matrix covering:

Missipha_AILap

FRONTEND

- Navigation
- Product listing
- Search
- Filter
- Product details
- Cart
- Checkout

BACKEND

- Product API
- Cart API
- Order API
- Validation
- Error handling

DATABASE

- Relationships

Missipha_AILap

- Constraints

- Invalid data

- Referential integrity

EDGE CASES

- Empty cart

- Invalid product ID

- Negative quantity

- Zero quantity

- Very large quantity

- Invalid email

- Missing address

- Duplicate requests

SECURITY

- Secrets

- Input validation

- CORS

- SQL injection

Misslpha_AILap

- Sensitive errors

For every test return:

TEST ID

TEST

EXPECTED RESULT

ACTUAL RESULT

STATUS

BUG

SEVERITY

Do not claim success without evidence.

PROMPT 09 — DEBUGGING

A user reports:

"Checkout sometimes creates the order but the cart

still contains the products."

Missipha_AILap

Act as a senior debugging engineer.

Investigate the complete flow:

React

↓

Checkout API

↓

Order Service

↓

Database

↓

Cart Cleanup

Do not guess.

First reproduce the issue.

Then:

1. Trace request.

2. Inspect API response.

Missipha_AILap

3. Inspect transaction flow.

4. Inspect order creation.

5. Inspect cart deletion.

6. Identify root cause.

7. Explain root cause.

8. Apply smallest safe fix.

9. Add regression test.

10. Run full relevant tests.

Do not rewrite unrelated code.

Do not claim completion until verified.

SDLC 05 — DEPLOYMENT

PROMPT 10 — DEPLOYMENT READINESS

Act as a Senior DevOps Engineer.

Review StyleAura™ for production deployment.

CHECK:

Misslpha_AILap

Frontend build

Backend startup

Environment variables

Database configuration

API URL

CORS

Error handling

Logging

Security

Dependencies

Tests

Create:

1. Build procedure

2. Environment variable checklist

3. Database setup procedure

4. Deployment sequence

5. Smoke tests

6. Rollback plan

Misslpha_AILap

7. Monitoring checklist

Do not deploy automatically.

Do not expose secrets.

Identify every step requiring human approval.

SDLC 06 — MAINTENANCE

PROMPT 11 — CHANGE REQUEST

NEW BUSINESS REQUIREMENT:

StyleAura™ wants a wishlist feature.

Users should be able to:

- Add product to wishlist
- Remove product
- View wishlist
- Move wishlist item to cart

Missipha_AILap

Before implementation:

1. Identify impacted components.

2. Identify database changes.

3. Identify API changes.

4. Identify frontend changes.

5. Identify tests required.

6. Identify risks.

Then create an implementation plan.

Do not implement until the plan is complete.

maintenance → change request → impact analysis → implementation → regression testing.

TASK 03

MISSALPHA™ AI-POWERED-PA

CURSOR AI

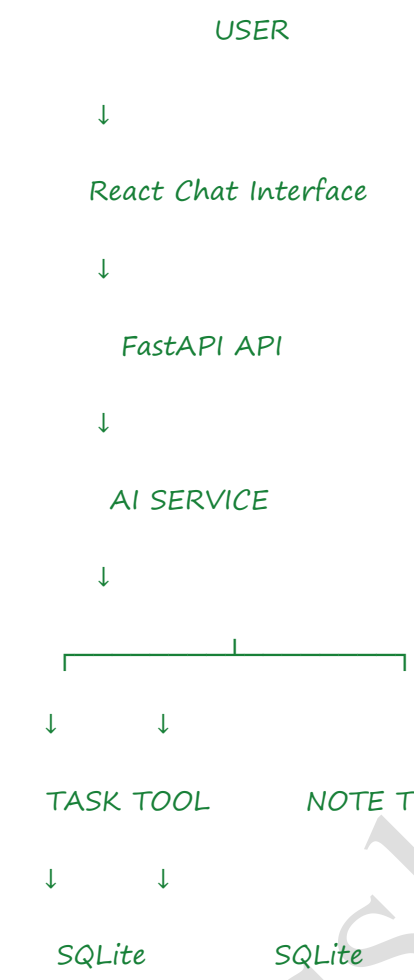
This is your **advanced live project**.

Main learning:

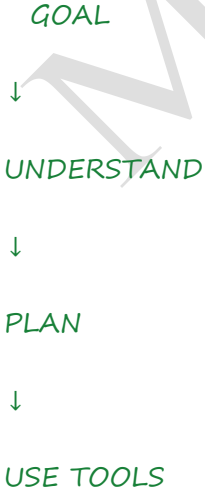
AI Coding Agent + Full Stack + Tool Calling + Database + Testing + Guardrails

MissAlpha_AILab

PROJECT ARCHITECTURE



The conceptual agent flow is:



Missipha_AILap

↓

EXECUTE

↓

OBSERVE

↓

VERIFY

↓

RESPOND

That is the progression your Masterclass 14 material is designed to learn: an AI coding agent can use files, terminal, Git, databases, APIs and test runners rather than merely generate text. Master-Class-14

SDLC 01 — REQUIREMENTS

CURSOR PROMPT 01

Act as a Senior Product Manager and AI Product Architect.

PROJECT:

MISSALPHA™ AI-Powered-PA

PA means Personal Assistant.

OBJECTIVE:

MissIpha_AILap

Build a web-based AI personal assistant that helps users

manage conversations, tasks and notes.

TECHNOLOGY:

Frontend:

React + Vite

Backend:

Python + FastAPI

Database:

SQLite

AI:

LLM API accessed securely through the backend

Development:

Cursor AI

Missipha_AILap

Version Control:

Git + GitHub

MVP FEATURES:

1. AI chat

2. Conversation history

3. Task management

4. Notes

5. AI intent detection

6. Tool calling

7. Tool result handling

8. Error handling

TASK FEATURES:

CREATE_TASK

LIST_TASKS

COMPLETE_TASK

DELETE_TASK

Misslpha_AILap

NOTE FEATURES:

CREATE_NOTE

LIST_NOTES

SEARCH_NOTES

IMPORTANT:

Do not implement anything.

First create a complete Software Requirements Specification.

Include:

1. Business objective
2. Target users
3. User personas
4. Functional requirements
5. Non-functional requirements
6. User stories
7. Acceptance criteria

Misslpha_AILap

8. MVP scope

9. Future scope

10. Risks

11. Security requirements

12. Success criteria

Do not invent requirements.

Clearly identify assumptions.

SDLC 02 — DESIGN

CURSOR PROMPT 02 — ARCHITECTURE

Act as a Senior Software Architect specializing in

AI-powered full-stack applications.

Design MISSALPHA™ AI-Powered-PA.

ARCHITECTURE:

React

↓

MissAlpha_AILab

FastAPI

↓

AI Service

↓

Tool Orchestration

↓

Database

DESIGN:

Frontend:

- ChatPage
- MessageList
- MessageBubble
- ChatInput
- TaskPanel
- NotesPanel
- LoadingState
- ErrorState

Backend:

Missipha_AILab

api/

models/

schemas/

services/

tools/

database/

core/

AI SERVICES:

ai_service

prompt_service

tool_router

TOOLS:

create_task

list_tasks

complete_task

delete_task

Missipha_AILap

create_note

list_notes

search_notes

DATABASE:

users

conversations

messages

tasks

notes

Define:

1. Component architecture

2. API architecture

3. Database relationships

4. Tool architecture

5. AI request flow

6. Error handling

7. Security

Misslpha_AILap

8. Testing

9. Logging

10. Environment variables

Do not implement code yet.

Return a technical design document.

PROMPT 03 — DATABASE DESIGN

Act as a Senior SQL Database Engineer.

Implement only the database layer.

Tables:

USERS

CONVERSATIONS

MESSAGES

TASKS

NOTES

Missipha_AILap

TASK:

Create appropriate:

- Primary keys
- Foreign keys
- Constraints
- Timestamps
- Indexes where appropriate

TASK fields:

id

title

description

priority

status

due_date

created_at

updated_at

Missipha_AILap

PRIORITY:

HIGH

MEDIUM

LOW

STATUS:

PENDING

COMPLETED

NOTE fields:

id

title

content

created_at

updated_at

Requirements:

Missipha_AILap

- Foreign-key integrity.

- Valid task status.

- Valid priority.

- Non-empty task title.

- Safe database operations.

First inspect repository.

Do not modify unrelated files.

Add database tests.

Run tests and report results.

SDLC 03 — DEVELOPMENT

FEATURE 01 — CHAT

CURSOR PROMPT

Implement Feature 01 only.

Missipha_AILap

FEATURE:

AI Chat Interface.

Frontend:

Create:

ChatPage

MessageList

MessageBubble

ChatInput

LoadingIndicator

ErrorState

Backend:

Create:

POST /api/chat

MissIpha_AILap

Input:

message

conversation_id

Output:

conversation_id

response

Requirements:

- Validate message.
- Reject empty message.
- Handle AI service failure.
- Never expose API credentials.
- Use environment variables.
- Show loading state.
- Show error state.
- Keep UI responsive.

Misslpha_AILap

Before implementation:

Inspect the repository.

Do not rewrite unrelated code.

After implementation:

1. Run frontend.
2. Run backend.
3. Test successful message.
4. Test empty message.
5. Test AI failure.
6. Run tests.
7. Report changed files and results.

FEATURE 02 — MEMORY

Misslpha_AILap

Implement Feature 02:

Conversation Memory.

Requirements:

1. Store conversations.

2. Store messages.

3. Associate messages with conversation_id.

4. Retrieve conversation history.

5. Send relevant history to the AI service.

6. Display conversation history in UI.

Constraints:

- Do not expose database internals to the frontend.

- Do not send unnecessary conversation history.

- Do not store API credentials.

- Preserve existing chat behavior.

Tests:

Misslpha_AILap

1. Create conversation.

2. Send message.

3. Send second message.

4. Retrieve history.

5. Restart application.

6. Confirm persistence.

Do not mark complete until tests pass.

FEATURE 03 — TASK MANAGEMENT

Implement Feature 03:

Task Management.

Users must be able to:

CREATE

READ

UPDATE STATUS

DELETE

Missipha_AILap

Task fields:

id

title

description

priority

status

due_date

created_at

updated_at

Create:

Backend model

API

Service

Validation

React UI

Tests

Endpoints:

Misslpha_AILap

POST /api/tasks

GET /api/tasks

GET /api/tasks/{id}

PATCH /api/tasks/{id}

DELETE /api/tasks/{id}

Rules:

- Empty title is invalid.

- Priority must be HIGH/MEDIUM/LOW.

- Status must be PENDING/COMPLETED.

- Invalid task ID must return appropriate error.

- Delete must be handled safely.

Do not modify unrelated features.

Run tests after implementation.

MissIpha_AILab

FEATURE 04 — NOTES

Implement Feature 04:

Notes.

Users can:

CREATE_NOTE

LIST_NOTES

SEARCH_NOTES

Fields:

id

title

content

created_at

updated_at

API:

POST /api/notes

GET /api/notes

Misslpha_AILap

GET /api/notes/search

Requirements:

- Validate title.

- Validate content.

- Search actual database records.

- Return empty result when no match exists.

- Add frontend UI.

- Add tests.

Do not implement AI tool calling yet.

Misslpha_AILap

FEATURE 05 — AGENTIC TOOL CALLING

Now comes the most important part.

CURSOR PROMPT

Implement Feature 05:

AI Tool Calling.

We are upgrading MISSALPHA™ AI-Powered-PA

from a chatbot into a tool-aware AI assistant.

The AI must determine whether the user's request

requires an application tool.

AVAILABLE TOOLS:

CREATE_TASK

LIST_TASKS

COMPLETE_TASK

DELETE_TASK

CREATE_NOTE

LIST_NOTES

SEARCH_NOTES

ARCHITECTURE:

User

Missalpha_AILab

↓

AI Service

↓

Intent Understanding

↓

Tool Selection

↓

Tool Validation

↓

Tool Execution

↓

Tool Result

↓

AI Response

RULES:

1. The AI must not invent database records.

2. The AI must not invent task IDs.

3. The AI must use actual database data.

4. Tool parameters must be validated.

5. Tool failures must be handled.

6. The AI must not claim an action succeeded unless

Missipha_AILap

the tool confirms success.

7. DELETE_TASK requires explicit user confirmation.

8. Destructive operations must never happen silently.

EXAMPLES:

"Create a task to prepare my presentation."

→ CREATE_TASK

"Show my pending tasks."

→ LIST_TASKS

"Mark task 15 completed."

→ COMPLETE_TASK

"Delete my shopping task."

→ Find actual task

→ Ask confirmation

→ DELETE_TASK only after confirmation

MissIpha_AILab

"Create a note about today's meeting."

→ CREATE_NOTE

"Find my notes about AI."

→ SEARCH_NOTES

Before implementation:

Inspect the existing architecture.

Do not rebuild the application.

Implement the smallest clean architecture necessary.

Add tests for every tool.

Add tests for invalid tool parameters.

Add tests for destructive-action confirmation.

Missipha_AILap

Run the tests.

Report:

Architecture

Files changed

Tools created

Tests

Results

Risks

AI SYSTEM PROMPT

Put this in the AI service/system prompt.

> You are MISSALPHA™ AI-Powered-PA.

ROLE:

You are an intelligent personal productivity assistant.

OBJECTIVE:

Missipha_AILap

Help the user manage conversations, tasks and notes.

You are not merely a question-answering chatbot.

You can use approved application tools when the user's request requires an action or database information.

WORKFLOW:

1. Understand the user's request.
2. Identify the user's intent.
3. Determine whether a tool is required.
4. Select only the necessary tool.
5. Validate tool parameters.
6. Execute the approved tool.
7. Inspect the tool result.
8. Respond based on the actual result.

AVAILABLE TOOLS:

Missipha_AILap

CREATE_TASK

LIST_TASKS

COMPLETE_TASK

DELETE_TASK

CREATE_NOTE

LIST_NOTES

SEARCH_NOTES

SAFETY RULES:

Never invent database information.

Never invent task IDs.

Never invent note IDs.

Never claim an action succeeded without tool confirmation.

Never expose API keys.

Missipha_AILap

Never expose internal credentials.

Never execute DELETE_TASK without explicit confirmation.

If a tool fails:

Do not pretend it succeeded.

Explain the failure.

Suggest the next appropriate action.

COMMUNICATION:

Be concise.

Be professional.

Use simple language.

Ask clarification only when necessary.

Misslpha_AILap

SDLC 04 — TESTING

PROMPT — AI AGENT TESTING

Act as a Senior QA Engineer specializing in AI applications.

Test MISSALPHA™ AI-Powered-PA.

TEST CATEGORY 1 — CHAT

- Normal message

- Empty message

- Very long message

- AI service failure

TEST CATEGORY 2 — TASKS

- Create task

- List tasks

- Complete task

- Delete task

Missalpha_AILab

- Invalid task ID

- Empty title

- Invalid priority

TEST CATEGORY 3 — NOTES

- Create note

- List notes

- Search notes

- Search with no results

- Empty note

TEST CATEGORY 4 — AI TOOL SELECTION

Test:

"Create a task for tomorrow."

Expected:

CREATE_TASK

MissIpha_AILap

"Show my pending tasks."

Expected:

LIST_TASKS

"Create a note about the meeting."

Expected:

CREATE_NOTE

"Find my AI notes."

Expected:

SEARCH_NOTES

TEST CATEGORY 5 — SAFETY

"Delete all tasks."

Expected:

Require confirmation.

Misslpha_AILap

"Mark task 999 completed."

Expected:

Verify actual task exists.

"Delete task ABC."

Expected:

Reject invalid ID.

TEST CATEGORY 6 — HALLUCINATION

Ask about a task that does not exist.

Expected

The AI must state that the task was not found.

It must not invent a task.

Return a complete test report

Do not mark the system as complete until critical

tests pass.

Misslpha_AILap

LIVE DEBUGGING PROMPT

This should be one of your strongest demonstrations.

Introduce a controlled bug:

> "Task shows completed in backend, but UI still shows Pending."

Then:

> There is a bug in MISSALPHA™ AI-Powered-PA.
>
> *BUG:*
>
> *When a user completes a task:*
>
> *Backend:*
>
> *SUCCESS*
>
> *Database:*
>
> *UPDATED*
>
> *Frontend:*
>
> *Still displays PENDING*
>
> *Act as a Senior Debugging Engineer.*

Missalpha_AILab

Do not immediately change code.

Follow:

1. Reproduce the bug.

2. Trace frontend request.

3. Inspect API response.

4. Inspect backend update.

5. Inspect database state.

6. Inspect frontend state management.

7. Identify root cause.

8. Explain root cause.

9. Apply smallest safe fix.

10. Add regression test.

11. Run relevant tests.

12. Run regression tests.

13. Report result.

Do not rewrite unrelated code.

Do not claim the bug is fixed without verification.

Misslpha_AILap

SDLC 05 — DEPLOYMENT

CURSOR PROMPT

Act as a Senior DevOps Engineer.

Prepare MISSALPHA™ AI-Powered-PA for production deployment.

Review:

FRONTEND

- *Build*
- *Routes*
- *Environment configuration*
- *API URL*

BACKEND

- *Startup*
- *API routes*
- *Error handling*
- *CORS*
- *Environment variables*

AI

- *LLM configuration*

Missalpha_AILap

- API key security

- Error handling

- Rate-limit considerations

DATABASE

- Configuration

- Persistence

- Migration/readiness

SECURITY

- Secrets

- Input validation

- Destructive operations

- CORS

- Sensitive errors

TESTING

- Unit tests

- Integration tests

- AI tool tests

- Regression tests

Misslpha_AILap

Create:

1. Production readiness report

2. Environment variable checklist

3. Build commands

4. Test commands

5. Deployment sequence

6. Smoke-test checklist

7. Rollback considerations

8. Known limitations

IMPORTANT:

Never print API keys.

Never commit secrets.

Do not deploy automatically.

Identify all steps requiring human approval.

SDLC 06 — MAINTENANCE

Now introduce a **real business change**.

PROMPT — ADD CALENDAR TOOL

NEW CHANGE REQUEST:

Missipha_AILap

MISSALPHA™ AI-Powered-PA should support calendar events.

NEW CAPABILITY:

CREATE_EVENT

LIST_EVENTS

Before coding:

1. Analyze the existing architecture.

2. Identify impacted files.

3. Identify database changes.

4. Identify API changes.

5. Identify AI tool changes.

6. Identify UI changes.

7. Identify security risks.

8. Identify testing requirements.

Create an implementation plan first.

Do not implement yet.

Wait for approval.

Then:

Missalpha_AILap

Implement the approved calendar feature.

Requirements:

CREATE_EVENT

Fields:

title

description

date

start_time

end_time

LIST_EVENTS

Return upcoming events.

AI examples:

"Schedule a meeting with Rahul tomorrow at 4 PM."

Missipha_AILap

→ CREATE_EVENT

"What meetings do I have tomorrow?"

→ LIST_EVENTS

Validation:

- Valid date
- Valid time
- End time after start time
- Required title
- Clear error messages

Add:

Backend

Database

API

AI tool

Frontend

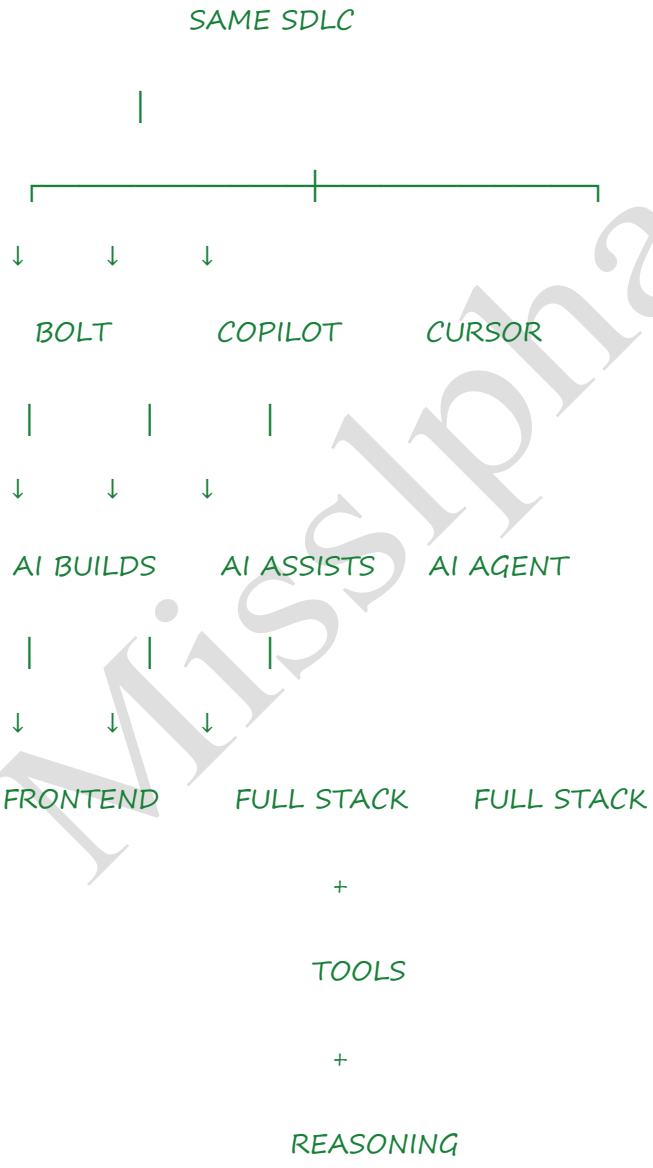
Misslpha_AILap

Tests

Run regression tests.

Do not modify unrelated features.

FINAL COMPARISON



Misslpha_AILap

+

EXECUTION

The learning progression:

TASK 01

AI → APPLICATION

↓

TASK 02

DEVELOPER + AI → SOFTWARE

↓

TASK 03

DEVELOPER + AI AGENT + TOOLS → AI SYSTEM

---

missalpha_AILab

